APLICACIONES WEB AVANZADAS

Laboratorio 04 Aplicación web N tier

Isabella Hernández

ASP.NET Web Forms y .NET Framework 4.8

Se implementaron cuatro proyectos independientes y se aprobaron 59 comprobaciones de código, SQL e interacción HTTP. El documento registra el código de Song, las pruebas y las capturas reales de la aplicación.

Componente	Estado
Solución	EntityLayer, DAL, BLL y WebAppPoliMusicV3.
Compilación Release	Cuatro DLL generadas para .NET Framework 4.8.
Pruebas aisladas	21 de 21 aprobadas.
Pruebas de BLL y DAL con SQL	17 de 17 aprobadas.
Pruebas funcionales HTTP	21 de 21 aprobadas contra la aplicación real.
Base	BDD_PoliMusic_Lab04, independiente de los laboratorios previos.
Capturas de la aplicación	Canciones reproduciendo Adventure y búsqueda sin resultados.
Publicación	Verificada en GitHub, rama main; captura remota incluida.

Estado de la entrega

Se entregan el código de las cuatro capas, la base independiente, los resultados de 59 comprobaciones y las capturas de ejecución y publicación. El código está disponible en GitHub y la carpeta Entregables contiene este PDF y sus evidencias.

1. Arquitectura y referencias

La solución SlnAppPoliMusic conserva la carpeta virtual WebAppPoliMusicV3_ASPX_F4.8_NLAYER. Cada capa se compila en su propia biblioteca. La presentación consulta BLL; BLL utiliza DAL; DAL ejecuta SQL parametrizado. User, Constants y Hash se comparten desde EntityLayer.

Proyecto	Referencias de proyecto	Contenido
EntityLayer.PoliMusicV3	Ninguna	User y Util/Constants, Util/Hash.
DAL.PoliMusicV3	EntityLayer.PoliMusicV3	CRUD de User, autenticación y consultas de Song.
BLL.PoliMusicV3	EntityLayer.PoliMusicV3 y DAL.PoliMusicV3	User, Song y RegistrationRules.
WebAppPoliMusicV3	EntityLayer.PoliMusicV3 y BLL.PoliMusicV3	Web Forms, Site.Master, sesión, archivos y audio.

La verificación por reflexión confirmó que la DLL Web referencia BLL y no DAL. También confirmó que EntityLayer no depende de BLL, DAL ni System.Web. Se corrigió la referencia circular aparente en la redacción de la guía: las referencias de negocio se configuran en BLL.

La carpeta Lab04_WebAppNTier está ubicada junto a Lab03_WebAppMonolitico y PoliMusic en el repositorio local. Se reutilizó Adventure y las imágenes del laboratorio anterior; las capas y la base del laboratorio 04 son independientes.

La guía menciona GitHub al inicio y Azure DevOps al final. Se publicó en el repositorio GitHub del curso ya utilizado. La comparación con su estado anterior confirmó que los cambios se limitan a Lab04_WebAppNTier.

2. Presentación y autenticación

Site.Master reúne la marca PoliMusic - isabella.hernandez, las opciones Inicio y Canciones, el usuario de sesión y Cerrar sesión. Default.aspx presenta la portada y enlaza a Song. Song.aspx, Default.aspx, DesktopAdmin.aspx y Logout.aspx usan la página maestra. Login y SignUp conservan formularios independientes.

La autenticación usa FormsAuthentication y una entidad User en sesión. AuthPage exige ambos datos para las páginas protegidas. Logout elimina la sesión y el ticket. Un usuario normal que solicita DesktopAdmin vuelve a Canciones. Se probaron la redirección anónima, las credenciales inválidas, el ingreso válido y la salida.

Las rutas amigables Default, Song, Login, SignUp y Logout se registran con MapPageRoute. ScriptManager y UpdatePanel implementan la búsqueda parcial. El diseño usa CSS y JavaScript locales. La búsqueda se comprobó en el navegador: un título inexistente produjo cero filas y el mensaje de estado vacío; Limpiar restauró Adventure. La reproducción avanzó hasta 0:39 de 3:07, con el botón de pausa visible.

Regla de registro	Verificación
Usuario	3 a 50 caracteres válidos; espacios externos normalizados.
Correo	Formato válido y máximo 50 caracteres.
Contraseña	8 a 128 caracteres y confirmación idéntica.
Fecha de nacimiento	Formato dd/MM/yyyy, fecha real, no futura y compatible con datetime.
Duplicados	Usuario y correo comprobados mediante BLL y SQL.
Foto opcional	Se usa default.jpg; las fotos personalizadas no se probaron en esta ejecución.

SHA1 se conserva para compatibilidad académica con la guía. Se incluyó la utilidad PBKDF2 con sal aleatoria. Los datos de registro usados para las pruebas son ficticios; el informe omite las contraseñas.

3. Código de acceso a datos de Song

DAL.PoliMusicV3/Song.cs

```
using System.Data;
using System.Data.SqlClient;
namespace DAL.PoliMusicV3
{
    public class Song
    {
        private readonly string connectionString;
        public Song(string strConnString) { connectionString=strConnString; }
        public DataTable Read() { return Read(string.Empty); }
        public DataTable Read(string search)
        {
            using(SqlConnection con=new SqlConnection(connectionString))
            using(SqlCommand cmd=new SqlCommand("SELECT ID_SONG,SONG_NAME,SONG_PATH,ISNULL(PLAYS,0) AS
PLAYS FROM dbo.TBL_SONG WHERE @SEARCH='' OR SONG_NAME LIKE @PATTERN ESCAPE '~' ORDER BY
SONG_NAME,ID_SONG",con))
            {
                cmd.Parameters.Add("@SEARCH",SqlDbType.NVarChar,100).Value=search;
                cmd.Parameters.Add("@PATTERN",SqlDbType.NVarChar,210).Value="%"+search.Replace("~","~~"
).Replace("%","~%").Replace("_","~_").Replace("[","~[")+"%";
                using(SqlDataAdapter adapter=new SqlDataAdapter(cmd)) { DataTable table=new
DataTable(); adapter.Fill(table); return table; }
            }
        }
        public string ReadPath(int id)
        {
            using(SqlConnection con=new SqlConnection(connectionString))
            using(SqlCommand cmd=new SqlCommand("SELECT SONG_PATH FROM dbo.TBL_SONG WHERE
ID_SONG=@ID",con))
            { cmd.Parameters.Add("@ID",SqlDbType.Int).Value=id; con.Open(); object
path=cmd.ExecuteScalar(); return path==null||path==System.DBNull.Value?null:path.ToString(); }
        }
    }
}
```

4. Código de negocio de Song

BLL.PoliMusicV3/Song.cs

```
using System.Data;
namespace BLL.PoliMusicV3
{
    public class Song
    {
        private readonly DAL.PoliMusicV3.Song data;
        public Song(string connectionString) { data=new DAL.PoliMusicV3.Song(connectionString); }
        public DataTable Read() { return data.Read(); }
        public DataTable Read(string search) { return
data.Read(search==null?string.Empty:search.Trim()); }
        public string ReadPath(int id) { return id>0?data.ReadPath(id):null; }
    }
}
```

Base de datos y recursos

Database/01_CrearBaseLaboratorio.sql crea BDD_PoliMusic_Lab04, TBL_USER y TBL_SONG si no existen. La inserción inicial de Adventure es idempotente. No modifica BDD_PoliMusic_Lab03 ni la base original. Se ejecutó correctamente contra el contenedor existente sql-server2025 en el puerto 14333 de la PC.

La conexión preparada en la PC se conserva en ConnectionStrings.local.config, cifrada con DataProtectionConfigurationProvider y excluida de Git. El paquete portable incluye LocalDB con autenticación integrada y un ejemplo para configurar otra instancia. No incluye la conexión real de la PC.

5. Diseño de Song.aspx

WebAppPoliMusicV3/Song.aspx

```
<%@ Page Title="Canciones" Language="C#" MasterPageFile="~/Site.Master" AutoEventWireup="true"
CodeBehind="Song.aspx.cs" Inherits="WebAppPoliMusicV3.Song" %>
<asp:Content ID="contentSong" ContentPlaceHolderID="MainContent" runat="server">
<section class="library-hero">
<div>
<span class="eyebrow">EL SOUNDTRACK DE TU DÍA</span>
<h1>Encuentra tu ritmo.<br />
<em>Hazlo tuyo.</em>
</h1>
<p>Todas tus canciones, en un solo lugar.<br />Descubre, elige y dale play.</p>
<a href="#songs" class="hero-button">Explorar canciones ↓</a>
</div>
<div class="hero-vinyl" aria-hidden="true">
<div>♪<small>POLIMUSIC</small>
</div>
</div>
<span class="hero-number">VOL. 04</span>
</section>
<asp:UpdatePanel ID="songsUpdate" runat="server" UpdateMode="Conditional"><ContentTemplate><section
id="songs" class="songs-section">
<div class="section-title">
<div>
<span class="eyebrow muted">DALE PLAY A LO QUE TE GUSTA</span>
<h2>Tu biblioteca <span class="count">
<asp:Literal ID="litCount" runat="server" />
</span>
</h2>
</div>
<div class="search-group">
<label class="sr-only" for="txtSearch">Buscar canción</label>
<asp:TextBox ID="txtSearch" runat="server" MaxLength="100" placeholder="Buscar una canción..." />
<asp:Button ID="btnSearch" runat="server" Text="Buscar" CssClass="search-button"
OnClick="btnSearch_Click" />
<asp:LinkButton ID="btnClear" runat="server" CssClass="clear-button"
OnClick="btnClear_Click">Limpiar</asp:LinkButton>
</div>
</div>
<asp:Label ID="lblMessage" runat="server" CssClass="message" role="alert" />
<div class="grid-wrap">
<asp:GridView ID="gvSongs" runat="server" AutoGenerateColumns="false" CssClass="song-grid"
GridLines="None" AllowPaging="true" PageSize="8" DataKeyNames="ID_SONG"
OnPageIndexChanging="gvSongs_PageIndexChanging" EmptyDataText="No hay canciones que coincidan con la
búsqueda." UseAccessibleHeader="true">
<Columns>
<asp:BoundField DataField="ID_SONG" HeaderText="#" />
<asp:TemplateField HeaderText="CANCIÓN">
<ItemTemplate>
<div class="track">
<span class="track-cover" aria-hidden="true">♫</span>
<div>
<strong>
<%# System.Web.HttpUtility.HtmlEncode(Convert.ToString(Eval("SONG_NAME"))) %>
</strong>
<small>Biblioteca PoliMusic · MP3</small>
</div>
</div>
</ItemTemplate>
</asp:TemplateField>
<asp:BoundField DataField="PLAYS" HeaderText="REPRODUCCIONES" />
<asp:TemplateField HeaderText="REPRODUCTOR">
<ItemTemplate>
<audio controls="controls" preload="none" aria-label='<%# AudioLabel(Eval("SONG_NAME")) %>' src='<%#
AudioUrl(Eval("ID_SONG")) %>'>Tu navegador no admite audio HTML5.</audio>
<span class="audio-error" role="status">
</span>
</ItemTemplate>
</asp:TemplateField>
</Columns>
<PagerStyle CssClass="pager" />
<PagerSettings Mode="NumericFirstLast" FirstPageText="Primera" LastPageText="Última" />
</asp:GridView>
</div>
<p class="library-note">Pausa, avanza o ajusta el volumen desde cada reproductor.</p>
</section>
</ContentTemplate></asp:UpdatePanel>
</asp:Content>
```

6. Code Behind y reproducción

WebAppPoliMusicV3/Song.aspx.cs

```
using System;
using System.Configuration;
using System.Data;
using System.Data.Common;
using System.Web;
using System.Web.UI.WebControls;
namespace WebAppPoliMusicV3
{
    public partial class Song : Util.AuthPage
    {
        protected void Page_Load(object sender,EventArgs e)
        {

            if(!IsPostBack) BindSongs();
        }
        private void BindSongs()
        {
            try
            {
                DataTable table=new BLL.PoliMusicV3.Song(ConfigurationManager.ConnectionStrings["BDD_Po
liMusicConnectionString"].ConnectionString).Read(txtSearch.Text);
                gvSongs.DataSource=table; gvSongs.DataBind();
litCount.Text=table.Rows.Count.ToString(); lblMessage.Text=string.Empty;
                if(gvSongs.HeaderRow!=null) gvSongs.HeaderRow.TableSection=TableRowSection.TableHeader;
            }
            catch(DbException) { gvSongs.DataSource=null; gvSongs.DataBind(); litCount.Text="0";
lblMessage.Text="No se pudo consultar la biblioteca. Revisa la conexión a SQL Server e intenta de
nuevo."; }
        }
        protected void btnSearch_Click(object sender,EventArgs e) { gvSongs.PageIndex=0; BindSongs(); }
        protected void btnClear_Click(object sender,EventArgs e) { txtSearch.Text=string.Empty;
gvSongs.PageIndex=0; BindSongs(); }
        protected void gvSongs_PageIndexChanging(object sender,GridViewPageEventArgs e) {
gvSongs.PageIndex=e.NewPageIndex; BindSongs(); }
        protected string AudioUrl(object id) { return
ResolveUrl("~/Media.ashx?id="+Convert.ToInt32(id).ToString()); }
        protected string AudioLabel(object name) { return HttpUtility.HtmlAttributeEncode("Reproducir
"+Convert.ToString(name)); }
    }
}
```

El GridView consulta Adventure desde SQL, permite buscar y limpiar el filtro, y tiene paginación de ocho filas. Cada fila integra un audio HTML5 con Media.ashx?id=ID_SONG. El manejador exige sesión, limita la ruta a SongFiles y admite rangos HTTP. La descarga directa desde SongFiles está bloqueada. PLAYS muestra el dato guardado; no se implementó un incremento automático.

Se verificaron una respuesta MPEG completa, una respuesta 206 de 100 bytes y una respuesta 416 para un rango inválido. Estas pruebas confirman la entrega del archivo. La captura del navegador muestra Adventure reproduciéndose a 0:39 de 3:07.

7. Resultados funcionales observados

Prueba	Resultado observado
Acceso anónimo protegido	Song redirige a Login
Login vacío	Validación de credenciales requeridas
Credenciales incorrectas	No se crea sesión
Confirmación diferente	Registro rechazado
Fecha imposible	Registro rechazado por BLL
Correo inválido	Registro rechazado por BLL
Registro con SQL Server	Cuenta normal creada desde SignUp
Usuario duplicado	Sin nueva cuenta duplicada
Correo duplicado	Sin nueva cuenta con el mismo correo
Login correcto y página Inicio	Inicio con Site.Master después de autenticar
GridView desde SQL	Adventure y usuario en la página Canciones
Página maestra personalizada	Marca, identidad y navegación común
Búsqueda positiva	Coincidencia recuperada por BLL → DAL
Búsqueda sin resultados	GridView presenta estado vacío
Filtro SQL parametrizado	El filtro se procesa como texto literal
Entrega del MP3	Audio MPEG servido desde Media.ashx
Rango HTTP del reproductor	206 Partial Content; 100 bytes recibidos
Rango inválido	416 Requested Range Not Satisfiable
Archivo de audio protegido	El acceso directo a SongFiles está bloqueado
Perfil normal restringido	Un usuario normal vuelve a Canciones
Logout invalida acceso	Después de salir se exige iniciar sesión

Las solicitudes recorrieron IIS Express, la capa web compilada, BLL, DAL y SQL Server. El registro se realizó desde SignUp.aspx como isabella.hernandez. Las búsquedas incluyeron una coincidencia, un título inexistente y un texto con sintaxis SQL.

8. Verificaciones de código y SQL

Pruebas aisladas

```
APROBADA: registro válido
APROBADA: normalizar usuario
APROBADA: usuario corto
APROBADA: usuario largo
APROBADA: usuario con ruta rechazado
APROBADA: correo inválido
APROBADA: límite del correo
APROBADA: contraseña corta
APROBADA: contraseña larga
APROBADA: confirmación diferente
APROBADA: fecha bisiesta normalizada
APROBADA: fecha imposible
APROBADA: fecha anterior a SQL datetime
APROBADA: fecha futura
APROBADA: fecha opcional
APROBADA: SHA1 compatible con la guía
APROBADA: PBKDF2 con sal aleatoria
APROBADA: entidad en DLL independiente
APROBADA: negocio en DLL independiente
APROBADA: Web referencia BLL y no DAL
APROBADA: entidades sin dependencia de infraestructura ni presentación
TOTAL: 21 pruebas aprobadas.
```

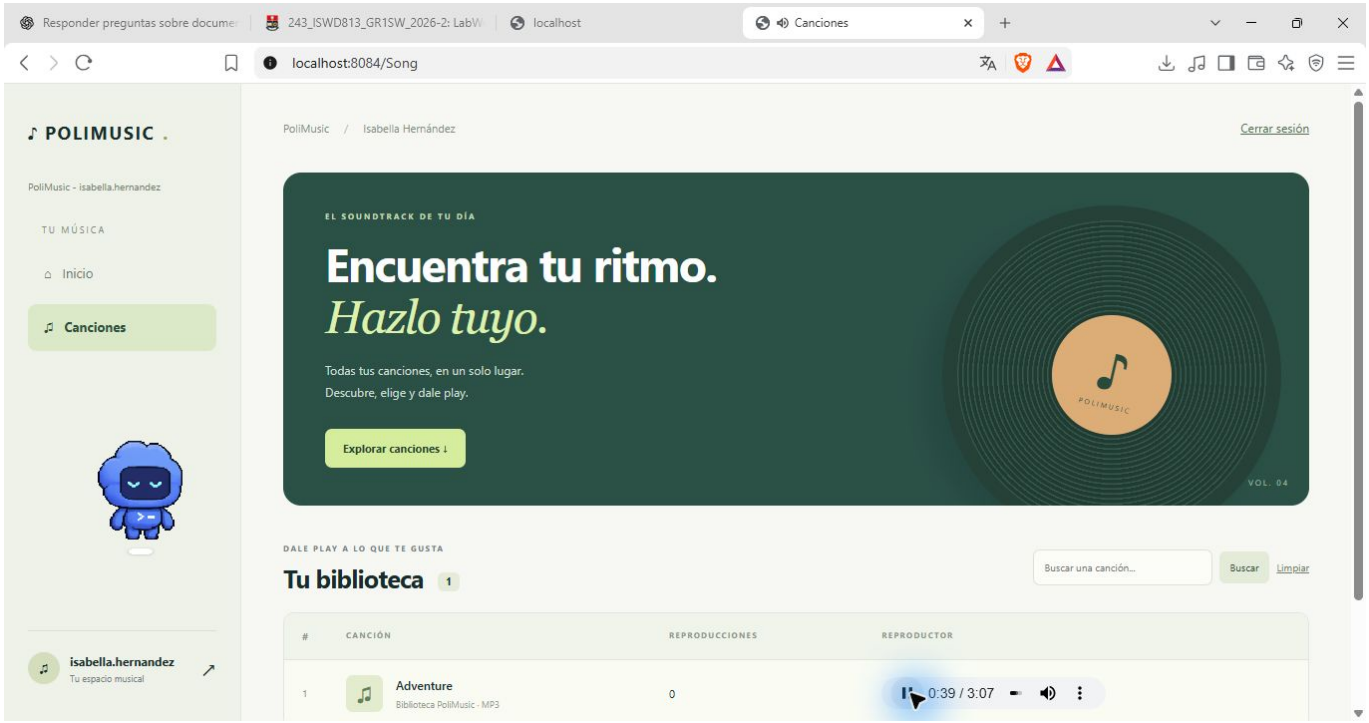
Pruebas con la base real

```
APROBADA: BLL y DAL insertan usuario con SQL
APROBADA: Consulta de usuario por nombre
APROBADA: Consulta de usuario por correo
APROBADA: BLL detecta duplicados
APROBADA: Autenticación válida con SQL
APROBADA: Autenticación inválida con SQL
APROBADA: Actualizar usuario conservando contraseña
APROBADA: Contraseña permanece válida tras actualizar
APROBADA: Lectura no expone contraseñas
APROBADA: Canción consultada mediante BLL y DAL
APROBADA: Filtro SQL parametrizado
APROBADA: LIKE trata porcentaje como literal
APROBADA: LIKE trata guion bajo como literal
APROBADA: BLL rechaza identificador de audio inválido
APROBADA: Ruta del audio desde SQL
APROBADA: Eliminar exclusivamente el usuario temporal de prueba
APROBADA: Confirmar eliminación del usuario temporal
TOTAL: 17 pruebas SQL aprobadas.
```

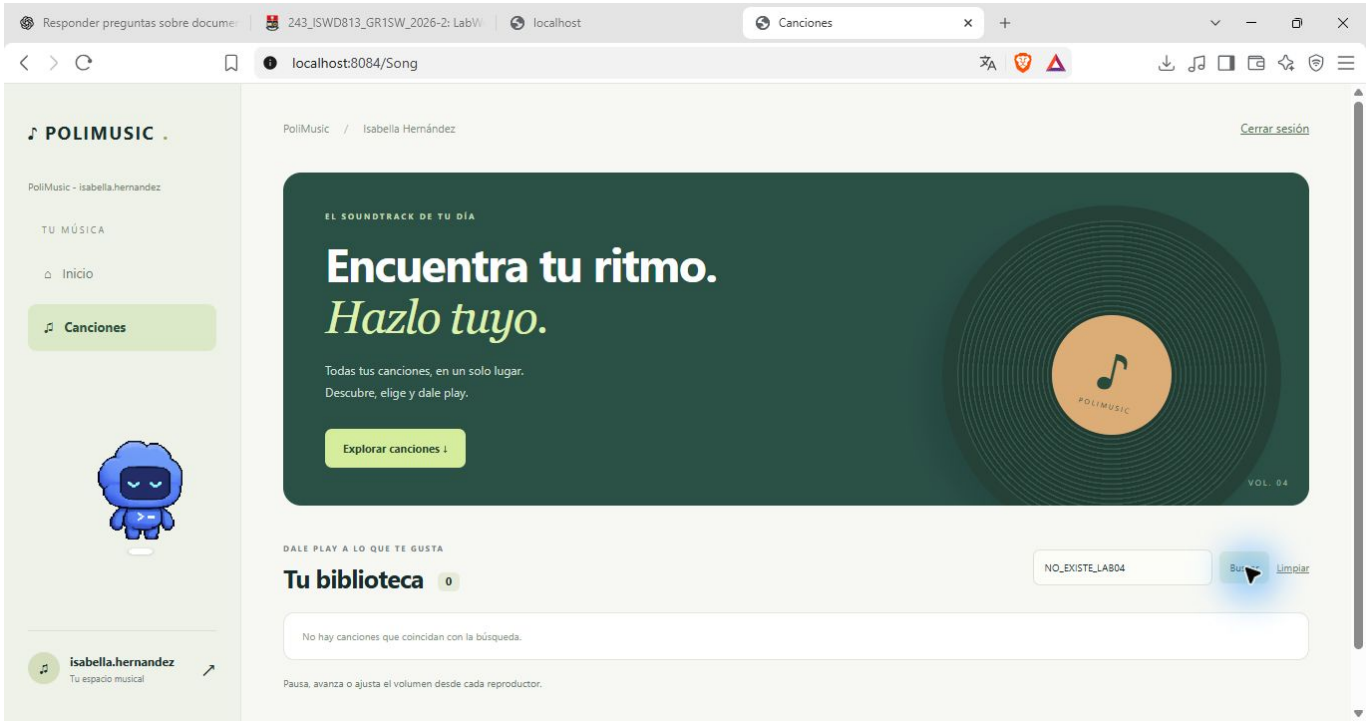
El ejecutor SQL creó un usuario temporal con nombre aleatorio, verificó inserción, lectura, actualización, autenticación y eliminación, y comprobó que su registro desapareció. No eliminó usuarios existentes. Los ejecutores reproducibles están incluidos en Verificacion.

9. Capturas de la aplicación

Canciones en ejecución. La captura muestra http://localhost:8084/Song, la marca PoliMusic - isabella.hernandez, el usuario de sesión, Adventure obtenido de SQL y el reproductor avanzando a 0:39 de 3:07. La aplicación se ejecuta desde Visual Studio en IIS Express con Clr4IntegratedAppPool.



Búsqueda sin coincidencias. El filtro NO_EXISTE_LAB04 produjo el contador cero y el mensaje de ausencia de coincidencias. La opción Limpiar restauró la biblioteca en la misma página.

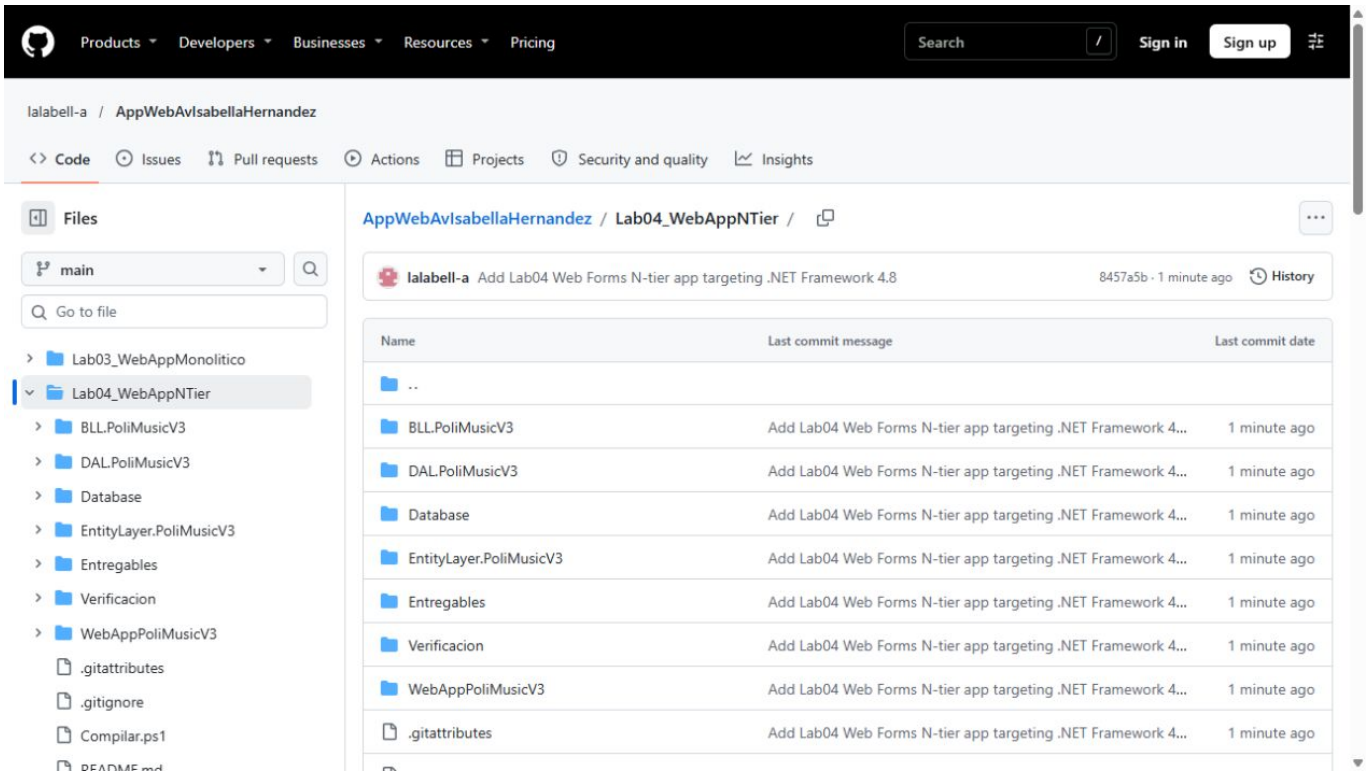


10. Publicación en GitHub

El código está publicado en la rama main. Se verificaron 71 archivos añadidos dentro de Lab04_WebAppNTier y cero cambios fuera de esa carpeta. PoliMusic y Lab03_WebAppMonolitico conservan su contenido.

Abrir Lab04_WebAppNTier en GitHub

Commit de publicación del código: 8457a5b0a8f3e0f57362b834531c8a6f40ed90fc



Captura real después del push. Se observan la rama main, la carpeta independiente y los cuatro proyectos de la solución. La captura corresponde al commit del código; este informe y su evidencia se incorporan en una actualización documental posterior.

Para ejecutar: abrir SlnAppPoliMusic.slnx, usar WebAppPoliMusicV3 como inicio y Default.aspx como página inicial, y ejecutar con Ctrl+F5 en IIS Express HTTP, puerto 8084. En otra PC, ejecutar el SQL inicial y configurar la conexión propia según README.md. Los ejecutores de pruebas están incluidos en Verificacion.

Referencia

Guía proporcionada LabWebAv04-ProyectoWebApp_NTier.docx. Se implementaron cuatro proyectos .NET Framework 4.8, User y utilidades, CRUD y autenticación, Song en DAL y BLL, página maestra, diseño propio de canciones, pruebas funcionales y evidencia de publicación.